Cursor AI Master Project Brief (Pragmatic Scope) v4.5

AI-Driven Social Media Audience Intelligence Framework

NTRO-Adjacent SOCMINT Framework | 7th Semester CSE Minor Project | Team of 4

> **Document Version:** v4.5 (Pragmatic Scope — Frontend Design Triad & Locked Agent Skills) | Aligned with Project Brief v4.5 & Architecture v4.5
> **Purpose:** This document is the pragmatic engineering and prompting brief for building the SOCMINT multi-agent framework inside **Cursor IDE** (Composer, Agent, and Chat).
> **Guiding Principle: Triage by Demo Need.** If the viva demo functions cleanly without a feature, that feature is omitted or deferred to "Future Work" in the report.

---

1. System Overview & Core Requirements

The platform monitors **5–10 public Telegram channels** around a strategic topic (e.g., ISRO/Indian space policy), collects public posts, runs five analytical agents (A–E), and surfaces findings through an interactive **7-view React dashboard** and a **grounded RAG query interface**.

Active Project Scope (What We Are Building):

1. **Component A (Ingestion):** Telethon MTProto client, 15-minute polling, rate-limit jitter, and offline demo seed replay script.
2. **Component B (Sentiment & Stance):** Language-routed GoEmotions/Indic-BERT pipeline, Zero-Shot NLI stance (`facebook/bart-large-mnli`, 0.60 threshold), and advisory sarcasm flag.
3. **Component C (Demographics):** 2-tier audience inference (channel metadata + discussion participant bios) with a $k \ge 50$ k-anonymity privacy filter.
4. **Component D (Trends):** BERTopic with multilingual MiniLM embeddings, ranking topics by velocity (posts/hr) on `posted_at`.
5. **Component E (Network & CIB):** NetworkX interaction graph, undirected Louvain community detection, and a clean **3-signal CIB anomaly average**.
6. **Orchestrator:** Simple Celery scheduling + an instant **"Run Analysis Now"** trigger for live viva demonstrations.
7. **RAG Query Layer:** Embedding dense 2–4 sentence agent summaries into pgvector (384-dim), cosine distance cutoff (< 0.65) with deterministic short-circuit, and audit logging to `rag_query_log`.
8. **Frontend Dashboard:** 7 clean views in React 18 + Tailwind CSS + Recharts + `react-force-graph`.

Explicitly Excluded (Deferred to "Future Work" in Report):

- ❌ **SHA-256 Evidentiary Hashing:** Platform-native message IDs handle deduplication; forensic hashing is documented as future production hardening.
- ❌ **Deletion Reconciliation Sweeps:** Ingestion polls forward-only; upstream deletion tracking is omitted.

- ❌ **Complex Salt-Hash Disposal Scripts & Compliance Tables:** Replaced with a simple post-viva SQL purge (`UPDATE raw_posts SET content_text = NULL; UPDATE raw_authors SET display_name = NULL, bio_text = NULL;`). Ethics sign-off is an administrative documentation point.
- ❌ **Dynamic Weighted CIB Math:** Replaced with an intuitive 3-signal arithmetic mean.
- ❌ **View 8 PDF Report Engine:** 7 interactive dashboard views are more than sufficient for evaluation.

---

2. Directory Layout & Module Structure

```
socmint-framework/
├── .cursorrules                # Cursor IDE automatic rules
├── CURSOR_BRIEF.md             # Master prompt reference (this file)
├── ARCHITECTURE.md             # Living architecture reference
├── README.md                   # Setup and runbook
├── docker-compose.yml          # postgres (pgvector), redis, celery, backend, frontend, ollama
├── .env.example
├── config/
│   └── channels.yaml           # Curated Telegram channels (seeded to Postgres on boot)
├── scripts/
│   └── seed_demo_data.sql      # 1,000 real historical posts for offline viva demo replay
│
├── storage/                    # Storage layer (SQLAlchemy ORM + Alembic)
│   ├── database.py             # Engine, sessionmaker
│   ├── migrations/             # Alembic migrations (0001 runs CREATE EXTENSION vector)
│   └── models/
│       ├── raw.py              # raw_posts, raw_authors, ingestion_runs, monitored_channels
│       ├── derived_b.py        # sentiment_results_b
│       ├── derived_c.py        # demographic_aggregates_c
│       ├── derived_d.py        # trend_topics_d
│       ├── derived_e.py        # network_nodes_e, network_edges_e
│       └── rag.py              # rag_documents, rag_query_log
│
├── ingestion_a/                # Component A: Ingestion Agent
│   ├── adapters/
│   │   ├── base.py             # BaseIngestionAdapter ABC
│   │   └── telegram.py         # Telethon adapter with rate-limiting and jitter
│   ├── backfill.py             # Offline seed replay script (--offline)
│   └── tasks.py                # Celery ingestion tasks
│
├── sentiment_b/                # Component B: Sentiment & Stance Agent
│   ├── agent.py                # SentimentAgent
│   ├── emotion.py              # GoEmotions / Indic-BERT language router
│   ├── stance.py               # Zero-shot NLI (facebook/bart-large-mnli)
│   ├── sarcasm.py              # Rule-based advisory sarcasm heuristic
│   └── summary.py              # 2–4 sentence summary generator for RAG
│
├── demographic_c/              # Component C: Demographic Agent
│   ├── agent.py                # DemographicAgent
│   ├── bio_parser.py           # Regex/keyword demographic inference
│   ├── cluster_guard.py        # k-anonymity guard (below_threshold if < 50)
│   └── summary.py
```

```
│
├── trend_d/                    # Component D: Trend & Topic Agent
│   ├── agent.py                # TrendAgent
│   ├── topic_model.py          # BERTopic + HDBSCAN sparse calibration (min_topic_size=3)
│   ├── velocity.py             # Velocity (posts/hr) on posted_at
│   └── summary.py
│
├── network_e/                  # Component E: Network Analysis & CIB Detection
│   ├── agent.py                # NetworkAgent
│   ├── graph_builder.py        # NetworkX directed graph from replies/forwards/mentions
│   ├── centrality.py           # Betweenness and degree centrality
│   ├── community.py            # Louvain modularity on undirected projection
│   ├── cib_signals.py          # 3-signal anomaly scorer (simple mean)
│   └── summary.py
│
├── orchestrator/               # Orchestration & Pipeline Runner
│   ├── coordinator.py          # Celery Beat schedules + manual trigger
│   └── aggregator.py           # Aggregates cross-component summary for /api/overview
│
├── rag/                        # RAG Grounding & Search
│   ├── embedder.py             # Embeds summaries using paraphrase-multilingual-MiniLM-L12-v2
│   ├── retriever.py            # pgvector search with < 0.65 cutoff
│   ├── prompt.py               # Grounded system prompt
│   └── llm/                    # Groq / Ollama / OpenAI provider abstraction
│
├── api/                        # FastAPI Backend
│   ├── main.py                 # App initialization, CORS, auth middleware
│   ├── auth.py                 # HTTP Basic Auth
│   ├── routers/                # overview, ingestion_a, sentiment_b, demographics_c,
│   │                           # trends_d, network_e, rag
│   └── schemas/                # Pydantic v2 schemas
│
└── frontend/                   # React 18 + Vite + Tailwind CSS Dashboard
    └── src/
        ├── views/              # Overview, SentimentTimeline, TrendExplorer, Demographics,
        │                       # NetworkGraph, DataSources, QueryPanel
        └── api/                # API client functions
```

---

3. Simplified Database Schema (PostgreSQL 16 + pgvector)

```
-- Initial migration prerequisite:
-- CREATE EXTENSION IF NOT EXISTS vector;

-- Component A: Ingestion execution runs
CREATE TABLE ingestion_runs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    started_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    completed_at TIMESTAMPTZ,
    platform VARCHAR(32) NOT NULL DEFAULT 'telegram',
    channel_ids JSONB,
    posts_collected INTEGER DEFAULT 0,
    status VARCHAR(32) DEFAULT 'running',
    error_message TEXT
);

-- Component A: Raw ingested posts
CREATE TABLE raw_posts (
    id BIGSERIAL PRIMARY KEY,
    platform VARCHAR(32) NOT NULL DEFAULT 'telegram',
    platform_post_id VARCHAR(255) NOT NULL,
    channel_id VARCHAR(255) NOT NULL,
    author_id VARCHAR(255),
    content_text TEXT,
    content_language VARCHAR(16),
    media_urls JSONB,
    reply_to_id VARCHAR(255),
    forward_from_id VARCHAR(255),
    view_count INTEGER,
    reaction_counts JSONB,
    ingested_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    posted_at TIMESTAMPTZ NOT NULL, -- ALL temporal metrics use this
    ingestion_run_id UUID REFERENCES ingestion_runs(id),
    processing_status VARCHAR(32) DEFAULT 'pending',
    UNIQUE(platform, platform_post_id)
);

-- Component A: Author metadata snapshot
CREATE TABLE raw_authors (
    id BIGSERIAL PRIMARY KEY,
    platform VARCHAR(32) NOT NULL DEFAULT 'telegram',
    platform_author_id VARCHAR(255) NOT NULL,
    display_name VARCHAR(512),
```

```
    bio_text TEXT,
    follower_count INTEGER,
    account_created_at TIMESTAMPTZ,
    snapshot_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    ingestion_run_id UUID REFERENCES ingestion_runs(id)
);

-- Component B: Derived sentiment, emotion & stance
CREATE TABLE sentiment_results_b (
    id BIGSERIAL PRIMARY KEY,
    raw_post_id BIGINT NOT NULL REFERENCES raw_posts(id),
    content_language VARCHAR(16) NOT NULL,
    dominant_emotion VARCHAR(64),
    emotion_confidence FLOAT,
    -- 6 core emotions (0.0 to 1.0)
    emotion_joy FLOAT,
    emotion_anger FLOAT,
    emotion_sadness FLOAT,
    emotion_disgust FLOAT,
    emotion_surprise FLOAT,
    emotion_neutral FLOAT,
    stance VARCHAR(32), -- 'supportive', 'against', 'neutral', 'unclear'
    stance_confidence FLOAT,
    sarcasm_flag BOOLEAN DEFAULT FALSE,
    valence VARCHAR(16), -- 'positive', 'negative', 'neutral'
    analyzed_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Component C: Demographic cluster distributions (aggregate only)
CREATE TABLE demographic_aggregates_c (
    id BIGSERIAL PRIMARY KEY,
    cluster_label VARCHAR(255) NOT NULL,
    window_start TIMESTAMPTZ NOT NULL,
    window_end TIMESTAMPTZ NOT NULL,
    account_count INTEGER NOT NULL,
    below_threshold BOOLEAN NOT NULL DEFAULT FALSE, -- TRUE if < 50
    age_bracket_dist JSONB,
    geography_dist JSONB,
    language_dist JSONB,
    interest_dist JSONB,
    inference_confidence FLOAT,
    analyzed_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Component D: Trend topics
```

```
CREATE TABLE trend_topics_d (
    id BIGSERIAL PRIMARY KEY,
    topic_label VARCHAR(512) NOT NULL,
    topic_keywords JSONB NOT NULL,
    window_start TIMESTAMPTZ NOT NULL,
    window_end TIMESTAMPTZ NOT NULL,
    post_count INTEGER NOT NULL,
    velocity FLOAT NOT NULL, -- posts/hour on posted_at
    acceleration FLOAT NOT NULL,
    rank INTEGER,
    sample_post_ids JSONB,
    analyzed_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Component E: Network graph nodes
CREATE TABLE network_nodes_e (
    id BIGSERIAL PRIMARY KEY,
    entity_id VARCHAR(255) NOT NULL,
    entity_type VARCHAR(32) NOT NULL, -- 'channel' or 'author'
    label VARCHAR(512) NOT NULL,
    window_start TIMESTAMPTZ NOT NULL,
    window_end TIMESTAMPTZ NOT NULL,
    degree_centrality FLOAT,
    betweenness_centrality FLOAT,
    community_id INTEGER,
    anomaly_score FLOAT, -- (s1 + s2 + s3) / 3
    posting_synchrony_signal FLOAT,
    age_volume_signal FLOAT,
    duplicate_content_signal FLOAT,
    coordination_signal BOOLEAN DEFAULT FALSE, -- TRUE if anomaly_score >= 0.70
    analyzed_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Component E: Network graph edges
CREATE TABLE network_edges_e (
    id BIGSERIAL PRIMARY KEY,
    source_entity_id VARCHAR(255) NOT NULL,
    target_entity_id VARCHAR(255) NOT NULL,
    interaction_type VARCHAR(64) NOT NULL, -- 'reply', 'forward', 'mention'
    weight FLOAT NOT NULL DEFAULT 1.0,
    window_start TIMESTAMPTZ NOT NULL,
    window_end TIMESTAMPTZ NOT NULL
);

-- RAG: Summary documents and embeddings
```

```
CREATE TABLE rag_documents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_component VARCHAR(8) NOT NULL, -- 'B', 'C', 'D', 'E'
    source_record_id TEXT NOT NULL,
    content_text TEXT NOT NULL,
    embedding vector(384),
    window_start TIMESTAMPTZ,
    window_end TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- RAG: Query audit log
CREATE TABLE rag_query_log (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    question_text TEXT NOT NULL,
    retrieved_doc_ids JSONB NOT NULL,
    llm_provider VARCHAR(64) NOT NULL,
    answer_text TEXT NOT NULL,
    grounded BOOLEAN DEFAULT TRUE,
    queried_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

---

4. Key Implementation Rules

1. Ingestion Rate Limiting & Concurrency:

- Telethon `.session` file is single-process: Celery worker for ingestion **must** run with `--concurrency=1`.
- Max 1 `GetHistory` call per channel per 30 seconds with 5–15s jitter.

2. Language & Emotion Routing:

- If Hindi in Devanagari (`re.search(r'[\u0900-\u097F]', text)`) $\rightarrow$ `vashuag/HindiEmotion` (exploratory; if load fails or errors occur, fallback to `cardiffnlp/twitter-xlm-roberta-base-sentiment`).
- Otherwise $\rightarrow$ `SamLowe/roberta-base-go_emotions` + `cardiffnlp/twitter-roberta-base-sentiment` .
- Compare only across shared core emotions (`joy` , `anger` , `sadness` , `disgust` , `surprise` , `neutral` , `fear`).

3. Zero-Shot Stance:

- `facebook/bart-large-mnli` against target topic.
- If both supportive and opposing probabilities < 0.60 $\rightarrow$ default to `'neutral'` .

4. Sarcasm as Advisory Penalty:

- Simple rule-based heuristic sets `sarcasm_flag = TRUE` and lowers confidence. **Never invert sentiment polarity.**

5. 3-Signal CIB Anomaly Detection:

- Simple, intuitive average: $$\text{anomaly_score} = \frac{s_1 + s_2 + s_3}{N_{\text{available}}}$$ where s_1 = posting synchrony, s_2 = account age vs volume, s_3 = content duplication (TF-IDF cosine).
- If $\text{anomaly_score} \ge 0.70 \rightarrow \text{coordination_signal} = \text{TRUE}$.

6. Network Graph Cap:

- Graph view capped at top 500 nodes by betweenness centrality. Edges strictly filtered: `source_id IN :top_500 AND target_id IN :top_500`.
- Louvain modularity uses `G.to_undirected(as_view=False)`.

7. RAG Grounding Guarantee:

- Vector search cutoff: `(embedding <=> :query) < 0.65`.
- If no documents qualify, return deterministic short-circuit: "*Insufficient data in current findings to answer this question.*"

---

5. Frontend Design Systems & UI/UX Agent Skills Suite

To ensure the Cursor Agent generates an authoritative, defense-grade operations console (rather than generic marketing slop or playful consumer SaaS cards), the agent must leverage the following design skills and specifications when building the frontend:

5.1 Design Skills Roster

Skill / Specification	Canonical Source	Role in This Project	Selection & Usage Directives for Cursor Agent
`DESIGN.md` **(Design System Tokens)**	VoltAgent/awesome-design-md	**Primary Design Ground Truth**	**Mandatory Baseline.** Cursor reads `DESIGN.md` (Linear Dark or Console preset) to lock design tokens: deep canvas (`#090d16`), card surfaces (`#0f172a`), 1px borders (`#1e293b`), and monospace metrics. Eliminates random Tailwind color hallucination.
`Web Design Guidelines`	vercel-labs/agent-skills	**Automated Quality Gate & Linter**	**Audit Stage.** After generating UI views, instruct Cursor to audit code against this skill to catch WCAG AA contrast violations, responsive breakpoint breaks, layout shifts, and table overflow issues.
`Taste Skill`	Leonxlnx/taste-skill	**Anti-AI-Slop & Density Guardrail**	**Scaffolding Stage.** Cursor uses this to avoid generic 3-column card templates and center-aligned landing page tropes. **Must tune parameters:** `VISUAL_DENSITY: 8/10` , `MOTION_INTENSITY: 2/10` , `DESIGN_VARIANCE: 4/10` (high data-ink ratio, zero bouncy spring animations).
`Image-to-Code Skill`	Leonxlnx/taste-skill/image-to-code-skill	**Mockup-to-Component Transpiler**	**Selective Use.** Use when translating reference operational dashboard mockups into pixel-perfect layout shells (e.g., top telemetry ribbon or RAG analyst query terminal).

5.2 Locked Design Language Triad & Operational Mandates

The frontend strictly operationalizes the following three design languages: 1. **Bento Grid (Structural Layout Blueprint)**: Modular, responsive, asymmetric rectangular cells that cleanly partition analytical agents (A–E) onto a unified tactical canvas without fragmented page-hopping. 2. **Dark Utilitarian Minimalism (Surface & Palette)**: High data-ink ratio inspired by Linear Dark / Datadog. Deep canvas (`#090d16`), card surfaces (`#0f172a`), 1px solid borders (`#1e293b`), monospace metrics (`JetBrains Mono` or `Consolas`), and zero decorative card slop. 3. **Spatial UI (Interactive Depth & HUD Elements)**: Z-index elevation layering, contextual floating drawers (for RAG citations and graph node inspections), and tactical HUD telemetry badges with subtle frosted glass accents (`backdrop-blur-md`). 4. **Color Roles**: Canvas `#090d16` , Surface `#0f172a` , Borders 1px solid `#1e293b` , Primary Cyan `#0ea5e9` , Support/Green `#10b981` , Alert/CIB Rose `#f43f5e` , Text `#f8fafc` . 5. **Motion Discipline**: Strictly subtle state transitions (150ms opacity fades only). Zero bouncing physics or sliding entrance delays that slow down intelligence analysts.

---

6. Phased Composer Prompts for Cursor

Use these prompts in **Cursor Composer** (Ctrl+I / Cmd+I) to generate the codebase sequentially:

Step 1: Storage Layer & Alembic Migrations

```
@storage/ @.cursorrules
Generate the SQLAlchemy 2.0 ORM models and Alembic migration for the SOCMINT framework.
Tables to implement: raw_posts, raw_authors, sentiment_results_b, demographic_aggregates_c, trend_topics_d,
Ensure storage/database.py exposes get_db() dependency and sessionmaker.
In migrations/versions/0001_initial.py, execute: op.execute("CREATE EXTENSION IF NOT EXISTS vector;") befor
```

Step 2: Component A Ingestion & Offline Seed Replay

```
@ingestion_a/ @storage/ @.cursorrules
Implement Component A (Ingestion Agent).
1. In ingestion_a/adapters/telegram.py: Telethon adapter polling public channels. Enforce rate limiting: <=
2. In ingestion_a/backfill.py: Implement offline demo replay script reading scripts/seed_demo_data.sql, set
3. In ingestion_a/tasks.py: Celery task to run ingestion for active channels.
```

Step 3: Component B Sentiment, Stance & Sarcasm

```
@sentiment_b/ @storage/ @.cursorrules
Implement Component B (Sentiment Agent).
1. Emotion routing: Devanagari regex ([\u0900-\u097F]) routes to vashuag/HindiEmotion (wrap in try/except;
2. Stance detection: Zero-shot NLI via facebook/bart-large-mnli with hard 0.60 threshold, defaulting to 'ne
3. Sarcasm: Heuristic flag (emoji-valence conflict) that sets sarcasm_flag = True and reduces confidence, b
4. Summary generator: 2-4 sentence summary string for RAG.
```

Step 4: Component D Trend & Velocity

```
@trend_d/ @storage/ @.cursorrules
Implement Component D (Trend Agent).
1. BERTopic topic modeling using paraphrase-multilingual-MiniLM-L12-v2 embeddings. Set min_topic_size = 3 a
2. Calculate topic velocity (posts/hr) and acceleration using raw_posts.posted_at.
3. Save topics to trend_topics_d.
```

Step 5: Component C Demographic Agent

```
@demographic_c/ @storage/ @.cursorrules
Implement Component C (Demographic Agent).
1. Extract aggregate language, geography, and interests from channel descriptions and discussion participan
2. Enforce k-anonymity guard: if account_count < 50, flag below_threshold = True.
3. Output to demographic_aggregates_c.
```

Step 6: Component E Network & 3-Signal CIB

```
@network_e/ @storage/ @.cursorrules
Implement Component E (Network Agent).
1. Build directed interaction graph (forwards, replies, mentions) in NetworkX.
2. Project to undirected graph (G.to_undirected(as_view=False)) and compute Louvain communities via python-
3. Compute 3-signal anomaly score: (posting_synchrony + age_volume + duplicate_content) / 3. Flag coordinat
4. Mask author labels: 'Actor-[Masked]-XXX'.
```

Step 7: Orchestrator & Live Demo Trigger

```
@orchestrator/ @storage/ @.cursorrules
Implement the Orchestrator.
1. Batch runner querying raw_posts WHERE processing_status = 'pending' and running Component B.
2. Window runner running Components C, D, and E over trailing 48 hours.
3. Provide a trigger_full_analysis() function callable via API endpoint /api/overview/trigger-analysis for
```

Step 8: RAG Grounding & Search

```
@rag/ @storage/ @.cursorrules
Implement the RAG Query Layer.
1. Embed agent summaries into rag_documents using paraphrase-multilingual-MiniLM-L12-v2 (384-dim).
2. Query pgvector cosine distance (<=>). Hard cutoff < 0.65; short-circuit with "Insufficient data..." if n
3. Grounding prompt with Groq (llama-3.3-70b) primary provider, Ollama (llama3.2:3b) offline provider, and
4. Log all queries to rag_query_log.
```

Step 9: FastAPI Backend

```
@api/ @storage/ @.cursorrules
Implement the FastAPI REST API with HTTP Basic Auth.
Routes:
- GET /api/overview/summary & POST /api/overview/trigger-analysis
- GET /api/ingestion-a/stats
- GET /api/sentiment-b/timeline
- GET /api/demographics-c/aggregates
- GET /api/trends-d/ranked
- GET /api/network-e/graph (capped at top 500 nodes by betweenness; strictly edge-induced)
- POST /api/rag/query
```

Step 10: React Intelligence Dashboard (7 Views via Design Skills Suite)

```
@frontend/ @api/ @.cursorrules @CURSOR_BRIEF.md
Build the React 18 + Vite intelligence dashboard adhering to the Locked Design Triad:
1. DESIGN TRIAD IMPLEMENTATION:
   - Layout: Bento Grid modular cell structure partitioning Agents A–E into a high-density operations cente
   - Aesthetic: Dark Utilitarian Minimalism (#090d16 canvas, #0f172a surface, 1px #1e293b borders, JetBrain
   - Depth: Spatial UI (floating HUD telemetry ribbon, z-index elevated RAG citation drawers, backdrop-blur
   - Taste Skill Parameters: VISUAL_DENSITY: 8/10, MOTION_INTENSITY: 2/10 (data-ink focus, zero bouncy anim
2. 7 CORE VIEWS:
   - Overview.jsx (Bento layout with KPI telemetry ribbon, pipeline status, "Run Analysis Now" viva trigger
   - SentimentTimeline.jsx (Recharts multilingual emotion trends, stance support/oppose gauges, advisory sa
   - TrendExplorer.jsx (BERTopic velocity [posts/hr] and acceleration ranked table with trend sparklines).
   - Demographics.jsx (Aggregate audience persona cards; if below_threshold=true, render grayed-out k-anony
   - NetworkGraph.jsx (Interactive Cytoscape / ForceGraph2D; top 500 nodes by betweenness; CIB anomaly clus
   - DataSources.jsx (Monitored Telegram channels, Telethon sync health, rate-limit jitter status).
   - QueryPanel.jsx (Analyst RAG terminal with in-text citation drawer and cosine distance confidence meter
3. POST-BUILD AUDIT:
   - Audit generated views against Vercel Web Design Guidelines (WCAG AA contrast, strict 8pt grid, zero ho
```

Document version: 4.5 (Cursor Master Brief — Design Skills Suite & Locked Triad) | Aligned with Project Brief v4.5 & Architecture v4.5 | September 2026